Advanced Python Interview Guide

Detailed advanced Python notes for production-ready coding and interviews

Detailed interview preparation and practice notes. Text only. No tables or images.

Use this PDF as source data for your RAG chatbot and as revision material before interviews.

1. Iterables and iterators

Concept

An iterable can return an iterator using iter(). An iterator returns items one by one using next(). Lists, tuples, strings, dictionaries, sets, and files are iterable.

Important details

Understanding iteration helps with loops, comprehensions, generators, and memory-efficient processing.

Common mistakes

Do not confuse iterable and iterator. A list is iterable, but iter(list) gives an iterator.

Interview answer style

Interview answer: iterator remembers state and produces next item until StopIteration.

Small code example

```
it = iter([1,2,3])
print(next(it))
```

Interview and practice focus

Be ready to explain iterables and iterators in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

2. Generators and yield

Concept

Generators produce values lazily using yield. They are memory efficient because they do not build the whole result at once.

Important details

Use generators for large files, streaming data, and pipelines where one item is processed at a time.

Common mistakes

Do not use return where yield is needed. Do not convert a huge generator to list unless necessary.

Interview answer style

Explain lazy evaluation and memory savings.

Small code example

```
def read_numbers():
    for i in range(3):
        yield i
```

Interview and practice focus

Be ready to explain generators and yield in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

3. Decorators

Concept

Decorators wrap functions to add behavior such as logging, timing, authentication, caching, and validation.

Important details

They are common in frameworks: @app.get in FastAPI, @st.cache_resource in Streamlit, @property in classes.

Common mistakes

Do not forget to return wrapper. Use functools.wraps to preserve metadata.

Interview answer style

Interview answer: decorators take a function, add behavior, and return a new function.

Small code example

```
from functools import wraps
```

Interview and practice focus

Be ready to explain decorators in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

4. Context managers

Concept

Context managers handle setup and cleanup using with. Files, locks, database sessions, and temporary resources use this pattern.

Important details

They reduce resource leaks and make cleanup guaranteed even if errors occur.

Common mistakes

Do not manually open files without closing them. Prefer with open.

Interview answer style

Explain __enter__ and __exit__ at advanced level.

Small code example

```
with open("a.txt", "r") as f:
    text = f.read()
```

Interview and practice focus

Be ready to explain context managers in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

5. Type hints and static checking

Concept

Type hints document expected types and help tools detect errors before runtime.

Important details

They improve maintainability in team projects and APIs. FastAPI and Pydantic use type hints heavily.

Common mistakes

Do not think type hints enforce types at runtime automatically.

Interview answer style

Interview answer: type hints improve readability, tooling, and API validation in frameworks.

Small code example

```
def ask(question: str, top_k: int = 3) -> str:
    return "answer"
```

Interview and practice focus

Be ready to explain type hints and static checking in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

6. Advanced function arguments

Concept

Python functions support positional-only, keyword-only, default arguments, *args, and **kwargs.

Important details

These features help design flexible APIs, but readability matters. Keyword-only arguments can make function calls clearer.

Common mistakes

Avoid mutable default values. Avoid overly flexible functions that accept anything.

Interview answer style

Explain *args collects positional arguments and **kwargs collects keyword arguments.

Small code example

```
def func(*args, **kwargs):
    print(args, kwargs)
```

Interview and practice focus

Be ready to explain advanced function arguments in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

7. Closures

Concept

A closure is a function that remembers variables from its enclosing scope even after the outer function has finished.

Important details

Closures are used in decorators, callbacks, and function factories.

Common mistakes

Do not confuse closure with class; both can preserve state, but in different ways.

Interview answer style

Interview answer: closure captures variables from enclosing scope.

Small code example

```
def outer(x):
    def inner(y): return x + y
    return inner
```

Interview and practice focus

Be ready to explain closures in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

8. Caching

Concept

Caching stores results so repeated work is avoided. Python has functools.lru_cache, and frameworks have their own caching tools.

Important details

In RAG apps, caching loaded vector DB or models prevents repeated expensive loading.

Common mistakes

Do not cache values that must always be fresh unless you have invalidation strategy.

Interview answer style

Explain caching as performance optimization by reusing previous results.

Small code example

```
from functools import lru_cache
@lru_cache
def get_config(): return {}
```

Interview and practice focus

Be ready to explain caching in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

9. Logging

Concept

Logging records application events with levels like DEBUG, INFO, WARNING, ERROR, and CRITICAL.

Important details

Use logging instead of print for production because logs can be formatted, filtered, stored, and monitored.

Common mistakes

Do not log secrets or sensitive user data.

Interview answer style

Interview answer: logging helps debug and monitor deployed apps.

Small code example

```
import logging
logging.info("started")
```

Interview and practice focus

Be ready to explain logging in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

10. Custom exceptions

Concept

Custom exceptions make errors meaningful and easier to handle.

Important details

Define exceptions such as DocumentLoadError, EmbeddingError, or VectorStoreError in larger projects.

Common mistakes

Do not raise generic Exception everywhere.

Interview answer style

Explain custom exceptions improve clarity and error handling.

Small code example

```
class VectorStoreError(Exception):
    pass
```

Interview and practice focus

Be ready to explain custom exceptions in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

11. Regular expressions advanced usage

Concept

Regex can extract groups, validate patterns, replace noisy text, and split complex strings.

Important details

Use named groups for readability and raw strings for patterns.

Common mistakes

Do not parse complex nested formats like HTML with regex.

Interview answer style

Interview answer: regex is suitable for pattern-based text extraction and cleaning.

Small code example

```
import re
match = re.search(r"(?P<name>\w+)", "Aman")
```

Interview and practice focus

Be ready to explain regular expressions advanced usage in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

12. Collections module

Concept

collections provides Counter, defaultdict, deque, namedtuple, and OrderedDict.

Important details

Counter counts occurrences, defaultdict handles missing keys, deque is efficient for queue operations.

Common mistakes

Do not use a list as a queue for frequent pop(0); deque is better.

Interview answer style

Mention Counter for word frequency and defaultdict for grouping.

Small code example

```
from collections import Counter
Counter("banana")
```

Interview and practice focus

Be ready to explain collections module in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

13. itertools module

Concept

itertools provides efficient iteration tools such as count, cycle, chain, combinations, permutations, product, islice, and groupby.

Important details

It helps build memory-efficient data transformations.

Common mistakes

Do not use itertools when a simple loop is clearer.

Interview answer style

Interview answer: itertools supports lazy iterator-based operations.

Small code example

```
from itertools import combinations
list(combinations([1,2,3], 2))
```

Interview and practice focus

Be ready to explain itertools module in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

14. functools module

Concept

functools includes lru_cache, partial, reduce, and wraps.

Important details

wraps is important in decorators, lru_cache improves repeated pure function calls, and partial pre-fills function arguments.

Common mistakes

Do not use reduce when sum or a simple loop is clearer.

Interview answer style

Explain functools as helper tools for functional programming and decorators.

Small code example

```
from functools import wraps, lru_cache
```

Interview and practice focus

Be ready to explain functools module in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

15. Async programming

Concept

async and await allow non-blocking I/O when using async-compatible libraries.

Important details

Useful for APIs, network calls, and database operations. It does not automatically speed up CPU-heavy tasks.

Common mistakes

Do not call blocking code directly inside async endpoints if performance matters.

Interview answer style

Interview answer: async improves handling of many waiting tasks, not CPU-heavy computation.

Small code example

```
async def fetch():
    return "data"
```

Interview and practice focus

Be ready to explain async programming in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

16. Threading

Concept

Threads can improve I/O-bound tasks where the program waits for network, disk, or external services.

Important details

Python has a GIL, so threads do not usually speed up CPU-bound Python code.

Common mistakes

Do not use threads without understanding shared state and race conditions.

Interview answer style

Explain threads for concurrent I/O.

Small code example

```
import threading
```

Interview and practice focus

Be ready to explain threading in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

17. Multiprocessing

Concept

Multiprocessing runs separate Python processes and can use multiple CPU cores.

Important details

It is useful for CPU-bound work but has overhead due to process creation and data transfer.

Common mistakes

Do not use multiprocessing for small tasks where overhead is larger than benefit.

Interview answer style

Mention it bypasses GIL by using separate processes.

Small code example

```
from multiprocessing import Pool
```

Interview and practice focus

Be ready to explain multiprocessing in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

18. Testing with pytest

Concept

pytest is a popular testing framework. It supports simple test functions, fixtures, parametrization, and assertions.

Important details

Writing tests for loaders, cleaners, chunkers, and retrievers makes the RAG project more reliable.

Common mistakes

Do not only test happy path; test empty input and error cases.

Interview answer style

Interview answer: tests prevent regressions and prove behavior.

Small code example

```
def test_clean_text():
    assert clean_text("a   b") == "a b"
```

Interview and practice focus

Be ready to explain testing with pytest in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

19. Packaging and dependency management

Concept

requirements.txt records dependencies. pyproject.toml is used by modern tools for packaging, formatting, testing, and build configuration.

Important details

Pinning versions improves reproducibility. Use virtual environments to isolate dependencies.

Common mistakes

Do not commit .venv. Do not include pip commands in requirements.txt.

Interview answer style

Explain how dependencies are installed during deployment.

Small code example

```
pip freeze > requirements.txt
```

Interview and practice focus

Be ready to explain packaging and dependency management in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

20. Environment variables

Concept

Environment variables store configuration and secrets outside source code.

Important details

Use os.getenv and python-dotenv locally. Deployment platforms provide secret settings.

Common mistakes

Do not expose tokens in GitHub, logs, PDFs, or screenshots.

Interview answer style

Interview answer: env variables separate configuration from code and protect secrets.

Small code example

```
import os
HF_TOKEN = os.getenv("HF_TOKEN")
```

Interview and practice focus

Be ready to explain environment variables in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

21. Performance profiling

Concept

Profiling measures where time is spent. Optimize only after measuring.

Important details

Use time, cProfile, logging timestamps, and memory tools. In RAG, common bottlenecks are document loading, embeddings, vector search, and LLM calls.

Common mistakes

Do not guess bottlenecks without evidence.

Interview answer style

Explain measure first, optimize second.

Small code example

```
import time
start = time.time()
```

Interview and practice focus

Be ready to explain performance profiling in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

22. Memory management and garbage collection

Concept

Python uses reference counting and garbage collection for cyclic references.

Important details

You normally do not manage memory manually, but you should avoid keeping unnecessary large objects in memory.

Common mistakes

Do not store huge intermediate lists if a generator or streaming approach works.

Interview answer style

Interview answer: Python automatically manages memory, but developers still design memory-efficient code.

Small code example

```
del large_object
```

Interview and practice focus

Be ready to explain memory management and garbage collection in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

23. Clean architecture for Python apps

Concept

Clean architecture separates UI/API, business logic, data access, and configuration.

Important details

This makes the same pipeline usable from CLI, Streamlit, FastAPI, and tests.

Common mistakes

Do not put all logic in app.py.

Interview answer style

Explain separation using app.py as entry point and src modules for logic.

Small code example

```
app.py -> src/rag_pipeline.py -> src/components.py
```

Interview and practice focus

Be ready to explain clean architecture for python apps in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

24. Advanced RAG engineering

Concept

A production-like RAG system handles document ingestion, cleaning, chunking, embedding model, vector store, retrieval strategy, prompt template, generation model, caching, error handling, and evaluation.

Important details

Good practices include creating embeddings once, loading DB once, top-k retrieval, source tracking, prompt grounding, and handling empty context.

Common mistakes

Do not hallucinate answers when context is missing. Tell the user when context does not contain the answer.

Interview answer style

Interview answer: RAG combines retrieval and generation to make answers more grounded in external documents.

Small code example

```
if not context:
    return "I do not know from the given context."
```

Interview and practice focus

Be ready to explain advanced rag engineering in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

25. Advanced interview practice plan

Concept

Practice explaining each concept, writing code from memory, debugging errors, and connecting concepts to your RAG chatbot.

Important details

For each topic, prepare definition, use case, syntax, common error, and one project example.

Common mistakes

Do not memorize without coding. Interviewers often ask follow-up questions based on your own project.

Interview answer style

Use your RAG project as proof of practical knowledge.

Small code example

```
print("Revise, code, explain, repeat")
```

Interview and practice focus

Be ready to explain advanced interview practice plan in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.